

Technical Specifications

Fecha: 2025-10-29

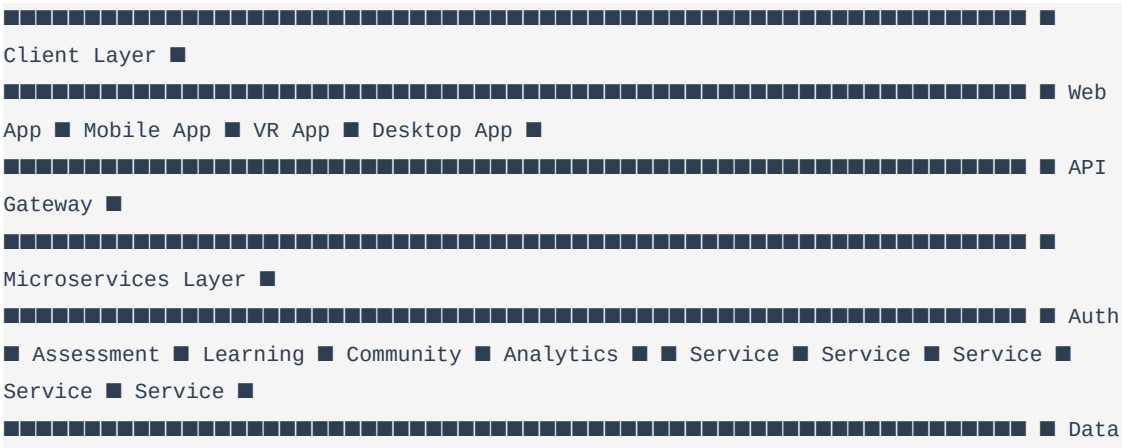
■ Neural Marketing Consciousness System - Technical Specifications

■ Table of Contents

- 1. [System Architecture](#system-architecture)
- 2. [Consciousness Assessment Platform](#consciousness-assessment-platform)
- 3. [Neural Marketing Toolkit](#neural-marketing-toolkit)
- 4. [Consciousness Development Dashboard](#consciousness-development-dashboard)
- 5. [Mobile Consciousness App](#mobile-consciousness-app)
- 6. [VR Consciousness Labs](#vr-consciousness-labs)
- 7. [API Documentation](#api-documentation)
- 8. [Database Schema](#database-schema)
- 9. [Security Specifications](#security-specifications)
- 10. [Performance Requirements](#performance-requirements)

System Architecture

High-Level Architecture





Technology Stack

Backend Services

- **Language:** Python 3.11+
- **Framework:** FastAPI 0.100+
- **Database:** PostgreSQL 15+ (Primary), MongoDB 6+ (Content)
- **Cache:** Redis 7+
- **Message Queue:** Celery with Redis
- **Search:** Elasticsearch 8+

Frontend Applications

- **Web App:** React 18+ with TypeScript
- **Mobile App:** React Native 0.72+
- **VR App:** Unity 2022.3+ with Oculus SDK
- **Desktop App:** Electron 25+

Infrastructure

- **Cloud Provider:** AWS
- **Containerization:** Docker + Kubernetes
- **CDN:** CloudFront
- **Monitoring:** DataDog + New Relic
- **CI/CD:** GitHub Actions

Consciousness Assessment Platform

Core Features

Assessment Engine

```
class ConsciousnessAssessment: def __init__(self): self.questions =  
self.load_questions() self.scoring_algorithm = ConsciousnessScoring()  
self.archetype_detector = ArchetypeDetector() def assess_consciousness(self, responses:  
List[Response]) -> AssessmentResult: # Calculate consciousness level (0-100%)  
consciousness_level = self.scoring_algorithm.calculate(responses) # Determine  
consciousness archetype archetype = self.archetype_detector.detect(responses) #  
Generate personalized recommendations recommendations =  
self.generate_recommendations(consciousness_level, archetype) return AssessmentResult(  
consciousness_level=consciousness_level, archetype=archetype,  
recommendations=recommendations )
```

Assessment Categories

1. **Technical Understanding** (25 questions)

- AI tool knowledge and proficiency
- Technical implementation skills
- Problem-solving abilities

2. **Strategic Thinking** (20 questions)

- AI marketing strategy development
- Long-term planning capabilities
- Innovation mindset

3. **Ethical Awareness** (15 questions)

- Responsible AI implementation
- Privacy and security considerations
- Bias prevention understanding

4. **Creative Application** (20 questions)

- AI-powered creative capabilities
- Brand voice development
- Content creation skills

5. **Leadership Readiness** (20 questions)

- Leading AI-conscious teams
- Change management abilities
- Vision and inspiration

Consciousness Archetypes

```
class ConsciousnessArchetype(Enum): VISIONARY = "visionary" # High transcendence, low technical ARCHITECT = "architect" # High technical, medium creativity EMPATH = "empath" # High emotional intelligence INNOVATOR = "innovator" # High creativity, medium empathy STRATEGIST = "strategist" # High analytical, medium creativity CATALYST = "catalyst" # High transformation, medium technical SAGE = "sage" # High wisdom, medium innovation PIONEER = "pioneer" # High exploration, medium stability HARMONIZER = "harmonizer" # High balance, medium specialization TRANSCENDENT = "transcendent" # High all dimensions SYNTHESIZER = "synthesizer" # High integration, medium depth ALCHEMIST = "alchemist" # High transformation, high creativity
```

Neural Marketing Toolkit

Tool Categories

Content Creation Tools

```
{ "content_tools": [ { "name": "GPT-4 Integration", "category": "text_generation",
"api_endpoint": "/api/tools/gpt4", "capabilities": ["content_writing", "copywriting",
"translation"], "consciousness_level_required": 20 }, { "name": "DALL-E Integration",
"category": "image_generation", "api_endpoint": "/api/tools/dalle", "capabilities":
["image_creation", "visual_content", "brand_assets"], "consciousness_level_required":
30 }, { "name": "Midjourney Integration", "category": "art_generation", "api_endpoint":
"/api/tools/midjourney", "capabilities": ["artistic_content", "creative_visuals",
"brand_art"], "consciousness_level_required": 40 } ] }
```

Analytics & Intelligence Tools

```
{ "analytics_tools": [ { "name": "Google Analytics AI", "category": "web_analytics",
"api_endpoint": "/api/tools/ga4", "capabilities": ["traffic_analysis",
"conversion_tracking", "audience_insights"], "consciousness_level_required": 25 }, {
"name": "Mixpanel Integration", "category": "product_analytics", "api_endpoint":
"/api/tools/mixpanel", "capabilities": ["user_behavior", "funnel_analysis",
"cohort_analysis"], "consciousness_level_required": 35 } ] }
```

Tool Integration Framework

```
class ToolIntegration:
    def __init__(self, tool_config: ToolConfig):
        self.tool_config = tool_config
        self.api_client = APIClient(tool_config.api_endpoint)
        self.consciousness_validator = ConsciousnessValidator()
    def execute_tool(self, user_id: str, tool_name: str, parameters: dict) -> ToolResult:
        # Validate consciousness level if not self.consciousness_validator.validate(user_id, tool_name):
        raise InsufficientConsciousnessError()
        # Execute tool with parameters
        result = self.api_client.execute(tool_name, parameters)
        # Log usage for analytics
        self.log_tool_usage(user_id, tool_name, result)
        return result
```

Consciousness Development Dashboard

Dashboard Components

Consciousness Level Visualization

```
interface ConsciousnessVisualization {
    currentLevel: number; // 0-100
    targetLevel: number; // 0-100
    progressPercentage: number; // 0-100
    archetype: ConsciousnessArchetype;
    skills: SkillProgress[];
    achievements: Achievement[];
    nextMilestone: Milestone;
}
interface SkillProgress {
    skillName: string;
    currentLevel: number;
    targetLevel: number;
    progressPercentage: number;
    lastUpdated: Date;
}
```

Real-Time Metrics

```
interface RealTimeMetrics {
    consciousnessLevel: number;
    learningVelocity: number; // Points per day
    streakDays: number; // Consecutive days
    totalLearningTime: number; // Minutes
    communityEngagement: number; // Interactions per day
    toolUsage: ToolUsageStats[];
}
```

Dashboard API Endpoints

```
@router.get("/dashboard/{user_id}") async def get_dashboard(user_id: str) -> DashboardData:
    """Get comprehensive dashboard data for user"""
    # Get consciousness
```

```

level and progress consciousness_data = await
consciousness_service.get_user_progress(user_id) # Get learning metrics
learning_metrics = await learning_service.get_metrics(user_id) # Get community
engagement community_data = await community_service.get_engagement(user_id) # Get tool
usage statistics tool_stats = await tool_service.get_usage_stats(user_id) return
DashboardData( consciousness=consciousness_data, learning=learning_metrics,
community=community_data, tools=tool_stats )

```

Mobile Consciousness App

App Architecture

React Native Structure

```

src/ ■■■ components/ # Reusable UI components ■■■ screens/ # App screens ■■■
services/ # API services ■■■ store/ # State management (Redux) ■■■ utils/ # Utility
functions ■■■ types/ # TypeScript type definitions ■■■ navigation/ # Navigation
configuration

```

Key Features

1. Daily Consciousness Check-ins

```

interface DailyCheckIn { date: string; consciousnessLevel: number; mood: MoodType;
energy: EnergyLevel; focus: FocusLevel; notes?: string; }

```

2. Micro-Learning Modules

```

interface MicroLearningModule { id: string; title: string; duration: number; // minutes
consciousnessLevel: number; type: ModuleType; content: ModuleContent; }

```

3. AI Chat Assistant

```

interface ConsciousnessChatBot { sendMessage(message: string): Promise<ChatResponse>;
getRecommendations(userId: string): Promise<Recommendation[]>;
scheduleReminder(reminder: Reminder): Promise<void>; }

```

Offline Capabilities

```

interface OfflineCapabilities { downloadContent: (moduleId: string) => Promise<void>;
syncProgress: () => Promise<void>; offlineMode: boolean; cachedContent:
CachedContent[]; }

```

VR Consciousness Labs

Unity VR Implementation

VR Environment Structure

```

public class ConsciousnessLab : MonoBehaviour { [SerializeField] private
ConsciousnessLevel currentLevel; [SerializeField] private LabEnvironment environment;
[SerializeField] private AIIntegration aiIntegration; public void

```

```
InitializeLab(ConsciousnessLevel level) { currentLevel = level;
environment.SetupEnvironment(level); aiIntegration.ConnectToConsciousnessAPI(); }
public void StartConsciousnessExercise(ExerciseType exercise) { var exerciseManager =
GetComponent<ExerciseManager>(); exerciseManager.StartExercise(exercise, currentLevel);
} }
```

VR Lab Environments

1. Consciousness Temple

- Sacred space for deep consciousness work
- Meditation and mindfulness exercises
- Energy visualization and manipulation

2. AI Collaboration Studio

- Virtual workspace for human-AI partnership
- Real-time AI interaction and feedback
- Collaborative creative sessions

3. Global Marketing Simulator

- Practice campaigns in virtual markets
- Multi-cultural marketing scenarios
- Real-time market response simulation

VR API Integration

```
public class VRConsciousnessAPI { private string apiEndpoint =
"https://api.neuralmarketingconsciousness.com/vr"; public async Task<ConsciousnessData>
GetConsciousnessLevel(string userId) { var response = await
httpClient.GetAsync($"{apiEndpoint}/consciousness/{userId}"); return await
response.Content.ReadFromJsonAsync<ConsciousnessData>(); } public async
Task<ExerciseResult> SubmitExerciseResult(string userId, ExerciseData data) { var
response = await httpClient.PostAsJsonAsync($"{apiEndpoint}/exercises", data); return
await response.Content.ReadFromJsonAsync<ExerciseResult>(); } }
```

API Documentation

Authentication

JWT Token Authentication

```
from fastapi import Depends, HTTPException, status from fastapi.security import
HTTPBearer import jwt security = HTTPBearer() async def get_current_user(token: str =
Depends(security)) -> User: try: payload = jwt.decode(token.credentials, SECRET_KEY,
algorithms=["HS256"]) user_id = payload.get("sub") if user_id is None: raise
HTTPException(status_code=401, detail="Invalid token") return await
user_service.get_user(user_id) except jwt.PyJWTError: raise
HTTPException(status_code=401, detail="Invalid token")
```

Core API Endpoints

Consciousness Assessment

```
@router.post("/assessment/start") async def start_assessment(user_id: str) ->
AssessmentSession: """Start a new consciousness assessment session""" session = await
assessment_service.create_session(user_id) return session
@router.post("/assessment/submit") async def submit_assessment( session_id: str,
responses: List[Response] ) -> AssessmentResult: """Submit assessment responses and get
results""" result = await assessment_service.process_responses(session_id, responses)
return result
```

Learning Management

```
@router.get("/learning/path/{user_id}") async def get_learning_path(user_id: str) ->
LearningPath: """Get personalized learning path for user""" path = await
learning_service.get_personalized_path(user_id) return path
@router.post("/learning/progress") async def update_progress( user_id: str, progress:
ProgressUpdate ) -> ProgressResult: """Update user learning progress""" result = await
learning_service.update_progress(user_id, progress) return result
```

Tool Integration

```
@router.post("/tools/execute") async def execute_tool( user_id: str, tool_name: str,
parameters: dict ) -> ToolResult: """Execute AI marketing tool""" result = await
tool_service.execute_tool(user_id, tool_name, parameters) return result
```

Database Schema

Core Tables

Users Table

```
CREATE TABLE users ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), email VARCHAR(255)
UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, first_name VARCHAR(100) NOT NULL,
last_name VARCHAR(100) NOT NULL, consciousness_level INTEGER DEFAULT 0, archetype
VARCHAR(50), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, updated_at TIMESTAMP
DEFAULT CURRENT_TIMESTAMP );
```

Consciousness Assessments

```
CREATE TABLE consciousness_assessments ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID REFERENCES users(id), session_id UUID NOT NULL, consciousness_level
INTEGER NOT NULL, archetype VARCHAR(50) NOT NULL, responses JSONB NOT NULL,
completed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

Learning Progress

```
CREATE TABLE learning_progress ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id
UUID REFERENCES users(id), module_id VARCHAR(100) NOT NULL, progress_percentage INTEGER
DEFAULT 0, completed_at TIMESTAMP, started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

MongoDB Collections

Content Library

```
// Content collection schema { _id: ObjectId, title: String, type: String, // webinar,
article, tool, exercise consciousness_level: Number, category: String, content: Object,
metadata: { duration: Number, difficulty: String, tags: [String], created_at: Date,
updated_at: Date } }
```

Community Interactions

```
// Community posts schema { _id: ObjectId, user_id: String, content: String, type:
String, // post, comment, question consciousness_level: Number, likes: Number,
comments: [ObjectId], created_at: Date, updated_at: Date }
```

Security Specifications

Authentication & Authorization

Multi-Factor Authentication

```
class MFAHandler: def __init__(self): self.totp = pyotp.TOTP() self.sms_service =
SMSService() async def setup_mfa(self, user_id: str) -> MFAConfig: secret =
self.totp.random_base32() qr_code = self.totp.provisioning_uri( user_id,
issuer_name="Neural Marketing Consciousness" ) return MFAConfig(secret=secret,
qr_code=qr_code) async def verify_mfa(self, user_id: str, token: str) -> bool:
user_secret = await self.get_user_mfa_secret(user_id) return self.totp.verify(token,
user_secret)
```

Role-Based Access Control

```
class RBACManager: ROLES = { "admin": ["read", "write", "delete", "manage_users"],
"instructor": ["read", "write", "manage_content"], "student": ["read", "write_own"],
"enterprise": ["read", "write", "analytics"] } async def check_permission(self,
user_id: str, action: str) -> bool: user_role = await self.get_user_role(user_id)
return action in self.ROLES.get(user_role, [])
```

Data Protection

Encryption at Rest

```
from cryptography.fernet import Fernet class DataEncryption: def __init__(self, key:
bytes): self.cipher = Fernet(key) def encrypt_sensitive_data(self, data: str) -> str:
return self.cipher.encrypt(data.encode()).decode() def decrypt_sensitive_data(self,
encrypted_data: str) -> str: return
self.cipher.decrypt(encrypted_data.encode()).decode()
```

API Rate Limiting

```
from slowapi import Limiter from slowapi.util import get_remote_address limiter =
Limiter(key_func=get_remote_address) @router.post("/assessment/submit")
@limiter.limit("10/minute") async def submit_assessment(request: Request, ...): #
Assessment submission logic pass
```

Performance Requirements

Response Time Targets

Scalability Requirements

- **Phase 1:** 10,000 concurrent users
- **Phase 2:** 50,000 concurrent users
- **Phase 3:** 100,000+ concurrent users
- **Phase 1:** 100GB initial storage
- **Phase 2:** 1TB with growth
- **Phase 3:** 10TB+ with archival
- **Phase 1:** 1,000 requests/second
- **Phase 2:** 5,000 requests/second
- **Phase 3:** 10,000+ requests/second

Monitoring & Alerting

```
class PerformanceMetrics:
    def __init__(self):
        self.response_times = []
        self.error_rates = []
        self.user_sessions = []
        self.api_calls = []
    def track_response_time(self, endpoint: str, duration: float):
        self.response_times.append({ "endpoint": endpoint, "duration": duration, "timestamp": datetime.now() })
    def track_error(self, endpoint: str, error: str):
        self.error_rates.append({ "endpoint": endpoint, "error": error, "timestamp": datetime.now() })
```

- Response time > 5 seconds: Warning
- Response time > 10 seconds: Critical
- Error rate > 5%: Warning
- Error rate > 10%: Critical
- CPU usage > 80%: Warning
- Memory usage > 90%: Critical